Fixing Clock Tree Propagation in the Common Clk Framework

Brian Masney
Principal Software Engineer
Red Hat, Inc.

Key questions to answer in this talk

- How do we build a consensus in a clock tree?
- Do we need to keep backwards compatibility with existing behavior?

Introduction

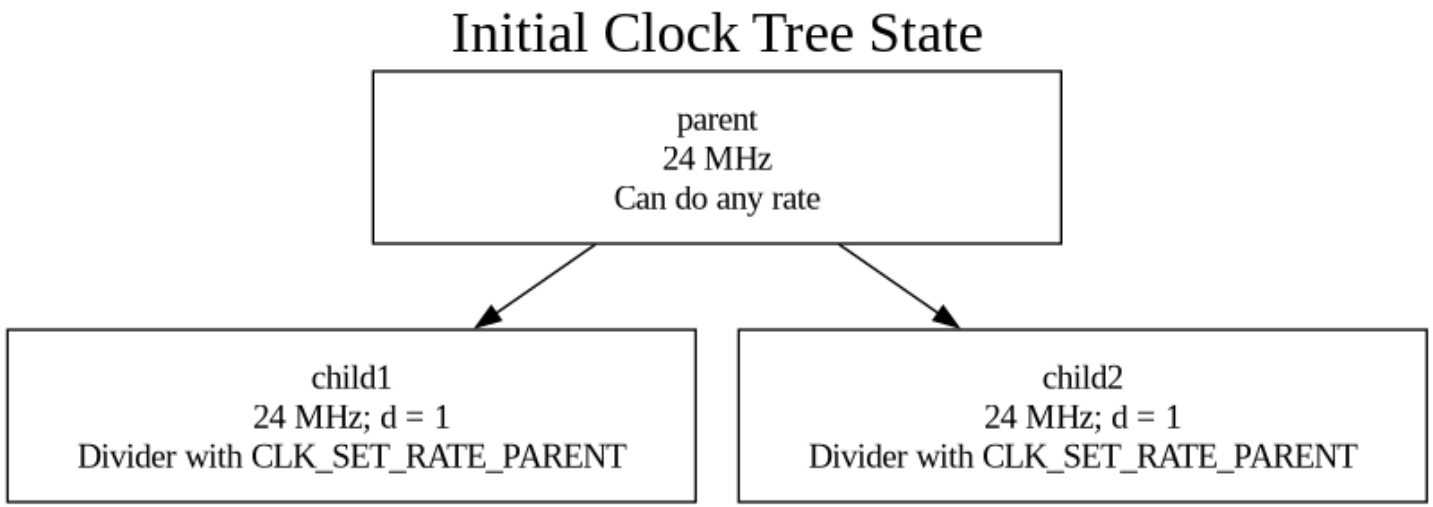
The Common Clk Framework (CCF) is expected to keep a clock's rate stable after setting a new rate with:

```
clk_set_rate(clk, NEW_RATE);
```

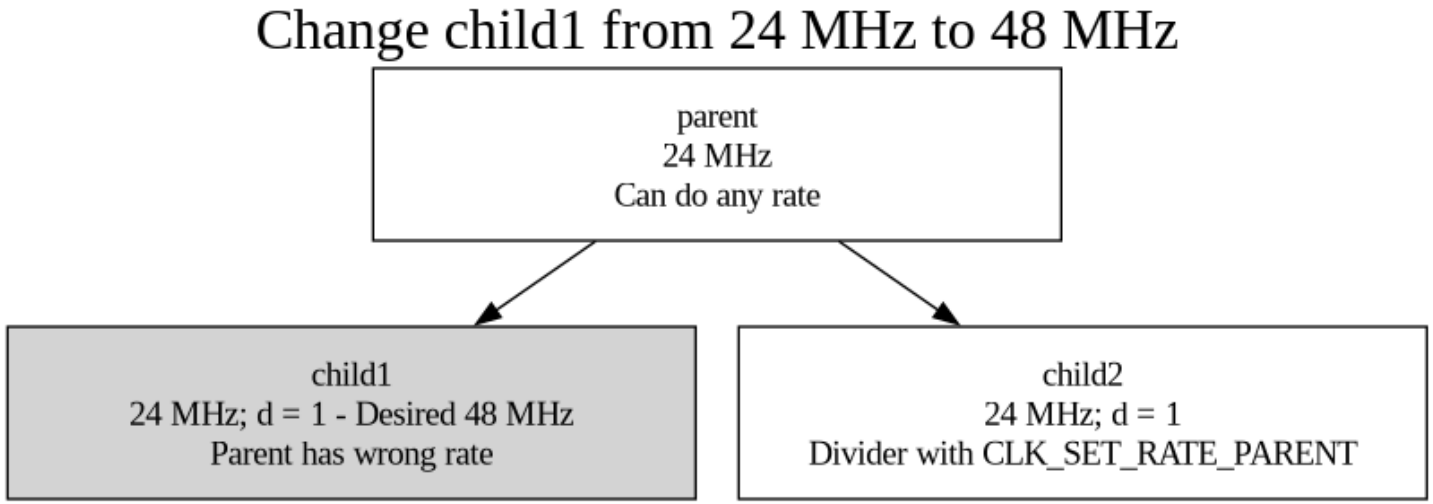
However, several longstanding issues affect how rate changes propagate through the clock tree when `CLK_SET_RATE_PARENT` is involved.

Consumers do not know about the clock hierarchy, sibling clocks, or the type of clocks involved.

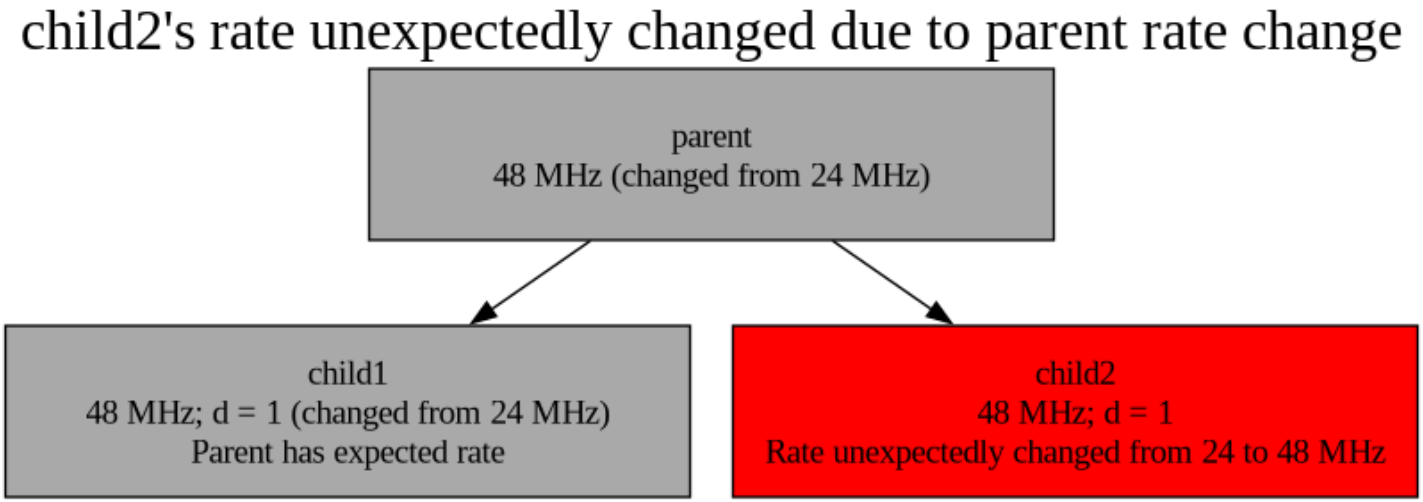
Problem 1: Rate Change That Affects a Sibling (1 / 4)



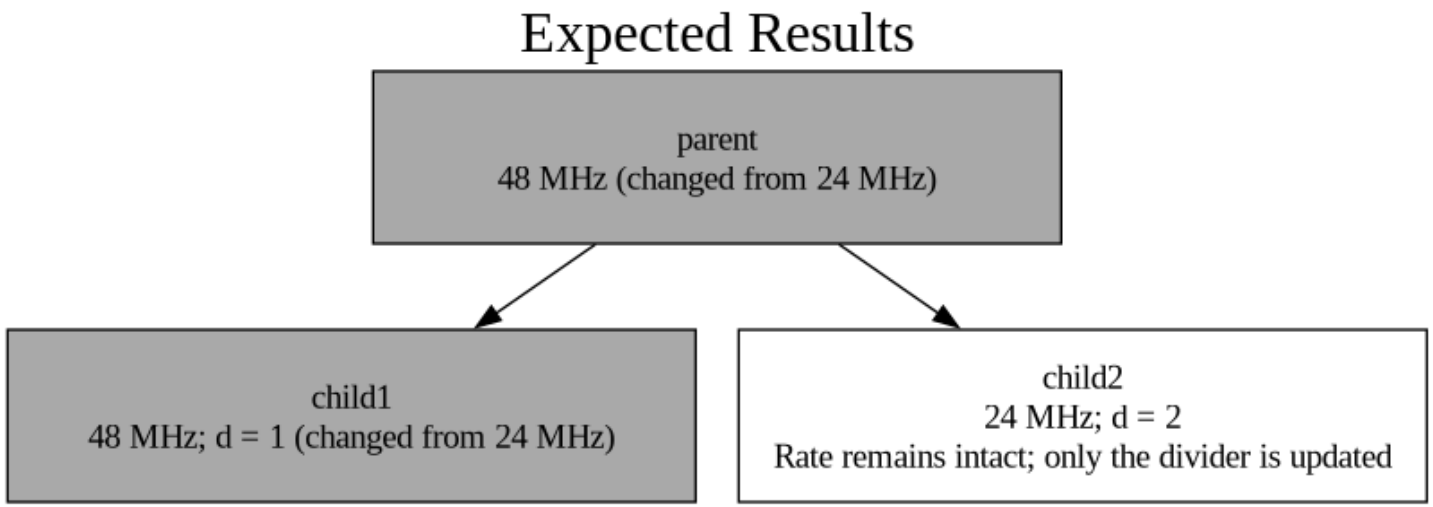
Problem 1: Rate Change That Affects a Sibling (2 / 4)



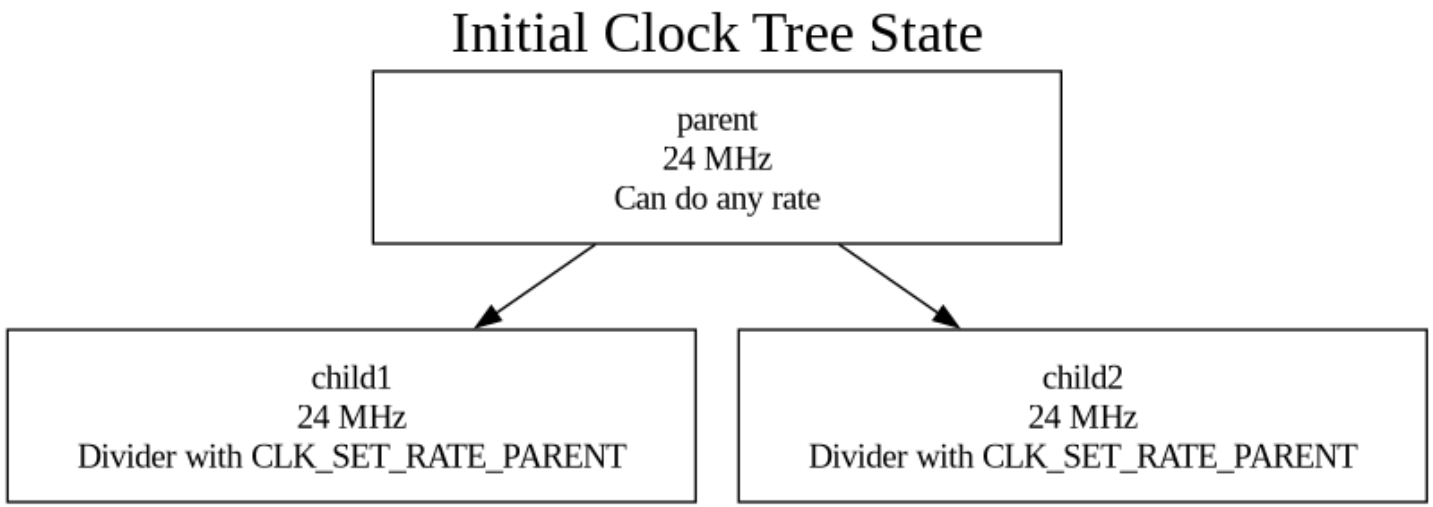
Problem 1: Rate Change That Affects a Sibling (3 / 4)



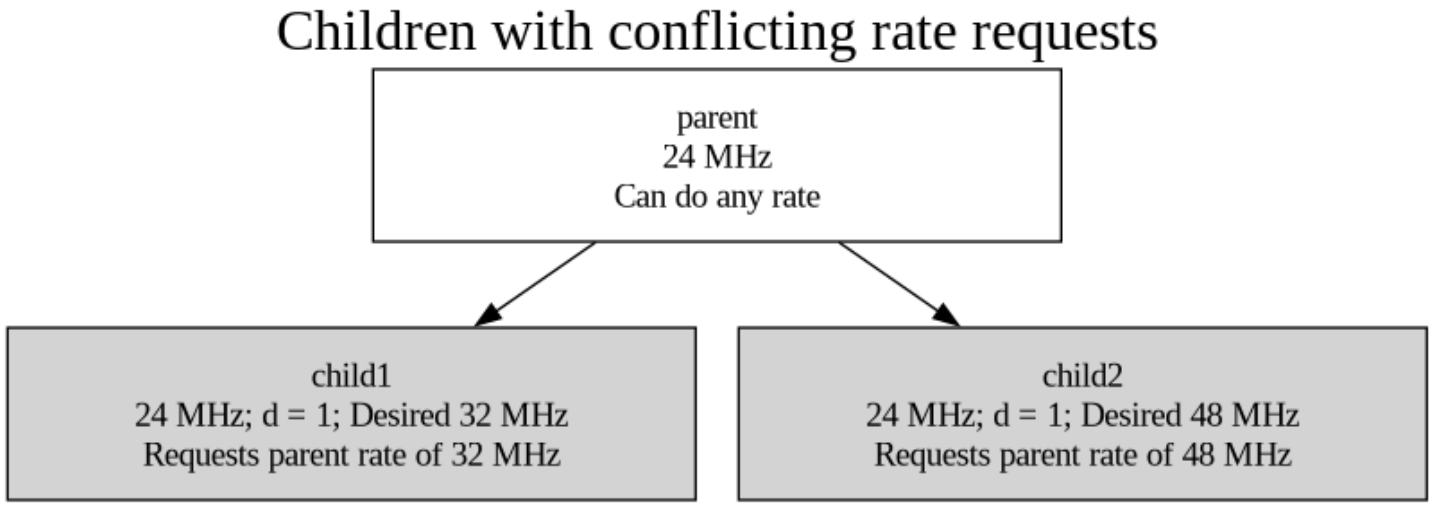
Problem 1: Rate Change That Affects a Sibling (4 / 4)



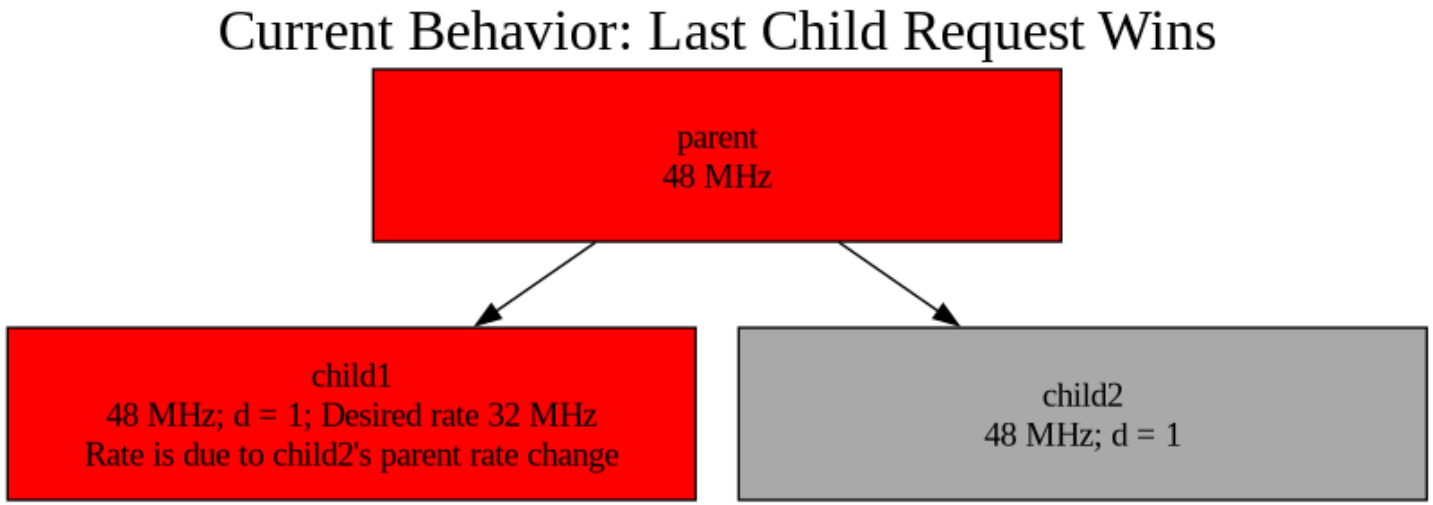
Problem 2: No parent rate negotiation with siblings (1 / 4)



Problem 2: No parent rate negotiation with siblings (2 / 4)

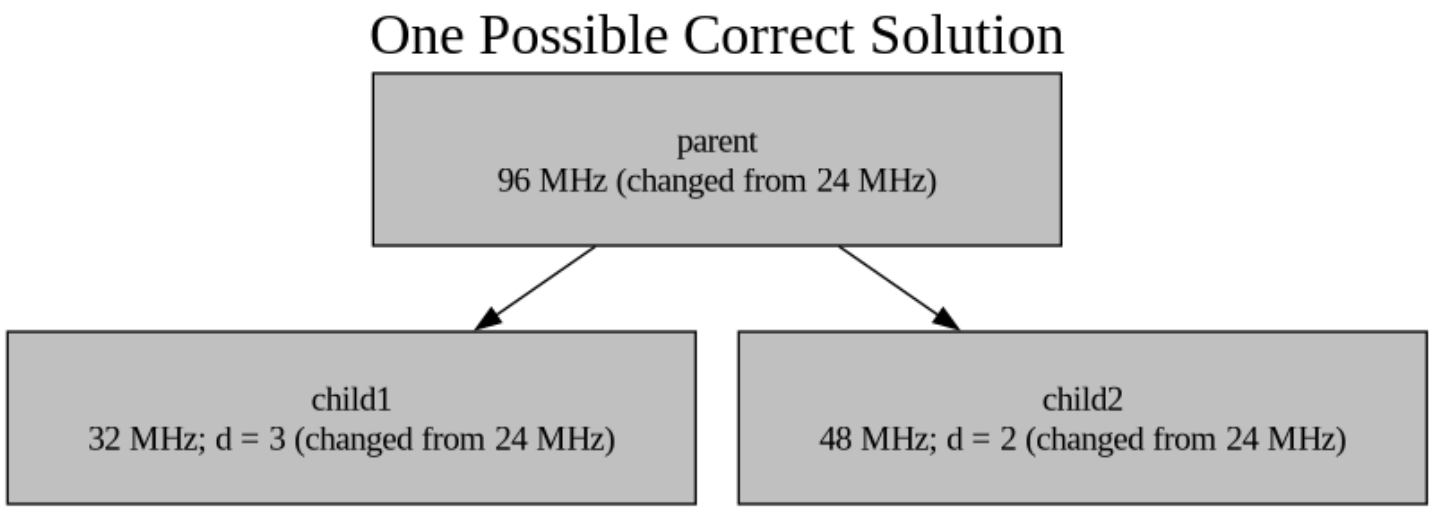


Problem 2: No parent rate negotiation with siblings (3 / 4)



The Common Clk's exclusive API doesn't solve the problem since it will lock the parent rate and prevent any changes. It can limit the rates that are possible in the tree.

Problem 2: No parent rate negotiation with siblings (4 / 4)



My proposed patch set linked at the end is able to successfully settle on this state.

High Level Proposed Solution

- Support both a v1 and a v2 rate negotiation logic in the clk core.
- Add a new member to `struct clk_ops` that allows the clk core to ask a provider about all of the possible rates for the v2 rate negotiation logic.
- The clk core starts at the top-most clk node that was identified that needs to change. Walks down the tree, and checks each of the proposed rates.
- The first rate that's acceptable to all nodes in the subtree is used.

Why not reuse `determine_rate` to avoid adding a new field to `struct clk_ops`?

- Stephen asked me to not add any new members to **`struct clk_ops`**.
- The clk core doesn't, and shouldn't, know about the internal state of the various clk providers.
- Child clks shouldn't have to know the internal state of the parent clks.
- Currently this information is not exposed in any way to the clk core.

v1 / v2 rate negotiation selection

- The v2 negotiation logic is only used if the `struct clk_ops` has a `negotiate_rates` member on `struct clk_ops` configured.
- Multiple clk providers can be used on a system, and allows for a mix of v1 and v2 negotiation logic in the tree.
- Introduce new kernel parameter `clk_v2_rate_negotiation` that defaults to enabled. This parameter is useful for debugging purposes to fall back to the v1 negotiation logic to see if that fixes any issues.

Outstanding Issue: Conversion still affects a large number of boards

- When shared drivers, such as `clk-fixed-factor.c` and `clk-divider.c` introduce a `negotiate_rates` op, a large number of boards will be eligible for v2 rate negotiation.
- In a similar fashion, when one vendor adds a `negotiate_rates` op, a large number of their systems move over at once.
- Could avoid this with a new flag in `struct clk_init_data`.
- We could default the `clk_v2_rate_negotiation` kernel parameter to off, and at some future period default it to on.
  - How to determine when to do that? Wait for one upstream LTS cycle to pass?
  - Will anyone have migrated by then?
  - What about the platforms where people want to move over to v2 logic?

Discussion

Code

- Most up to date code is available at: https://lore.kernel.org/linux-clk/20250923-clk-tests-docs-v4-0-9205cb3d3cba@redhat.com/T/

Examples of real world problems

This issue is encountered in real world scenarios. A sampling of some semi-recent workarounds caused by this issue:

- https://lore.kernel.org/lkml/20230825-pll-mipi_keep_rate-v1-0-35bc43570730@oltmanns.dev/
- https://lore.kernel.org/linux-kernel/20230807-pll-mipi_set_rate_parent-v6-0-f173239a4b59@oltmanns.dev/
- https://lore.kernel.org/all/20241114065759.3341908-1-victor.liu@nxp.com/
- https://lore.kernel.org/linux-clk/20241121-ge-ian-debug-imx8-clk-tree-v1-0-0f1b722588fe@bootlin.com/

DRM needs to run at precise clock rates, and this issue is encountered there. This is not isolated to just the DRM subsystem.

Problem 1: Can be fixed with the following, however...

```
diff --git a/drivers/clk/clk.c b/drivers/clk/clk.c
index a130eac9072dc7e71f840a0edf51c368650f8386..65408899a4ae8674e78494d77ff07fa658f7d3b0 100644
--- a/drivers/clk/clk.c
+++ b/drivers/clk/clk.c
@@ -2306,7 +2311,11 @@ static void clk_calc_subtree(struct clk_core *core,
                                                unsigned long new_rate,
     hlist_for_each_entry(child, &core->children, child_node) {
-        child->new_rate = clk_recalc(child, new_rate);
+        if (THIS_CHILD_NODES_RATE_WAS_CHANGED)
+            child->new_rate = clk_recalc(child, new_rate);
+        else
+            child->new_rate = child->rate;
+
         clk_calc_subtree(child, child->new_rate, NULL, 0);
     }
```

More details available at: https://lore.kernel.org/linux-clk/20250528-clk-wip-v2-v2-2-0d2c2f220442@redhat.com/

However, we can't easily fix it because an unknown subset of existing boards are unknowingly dependent on the existing behavior.

Example parent rate change

Example code from `drivers/clk/clk-fixed-factor.c` that shows what's typically done based on the current constraints within the framework.

```
static int clk_factor_determine_rate(struct clk_hw *hw,
                                     struct clk_rate_request *req)
{
        struct clk_fixed_factor *fix = to_clk_fixed_factor(hw);

        if (clk_hw_get_flags(hw) & CLK_SET_RATE_PARENT) {
                unsigned long best_parent;

                best_parent = (req->rate / fix->mult) * fix->div;
                req->best_parent_rate = clk_hw_round_rate(clk_hw_get_parent(hw), best_parent);
        }

        req->rate = (req->best_parent_rate / fix->div) * fix->mult;

        return 0;
}
```

We can test for these scenarios with kunit

```
/*
 * The test setup for clock tree not shown for simplicity. The clocks parent,
 * child1, and child2 are all initially 24 MHz, and the two children have the
 * flag CLK_SET_RATE_PARENT enabled. The parent is a dummy clock able to achieve
 * any desired rate.
 */

// Change child1 from 24 to 48 MHz
ret = clk_set_rate(ctx->child1_clk, 48 * HZ_PER_MHZ);
KUNIT_ASSERT_EQ(test, ret, 0);

// Parent rate should be changed from 24 to 48 MHz
KUNIT_EXPECT_EQ(test, clk_get_rate(ctx->parent_clk), 48 * HZ_PER_MHZ);

// Child1 should be changed from 24 to 48 MHz
KUNIT_EXPECT_EQ(test, clk_get_rate(ctx->child1_clk), 48 * HZ_PER_MHZ);
KUNIT_EXPECT_EQ(test, ctx->child1.div, 1);

/*
 * Child 2 should retain it's existing rate of 24 MHz. The clock's divider
 * should be changed from 1 to 2. These checks currently fail since child2's
 * rate is unexpectedly changed to 48 MHz, and retains it's existing
 * divider of 1.
 */
KUNIT_EXPECT_EQ(test, clk_get_rate(ctx->child2_clk), 24 * HZ_PER_MHZ);
KUNIT_EXPECT_EQ(test, ctx->child2.div, 2);
```

New op added to `struct clk_ops` on clk providers

When a child clk requests a new rate that requires a rate change from the parent, this negotiates a new parent rate that's acceptable to all of the children, and all the way down the subtree.

```
bool (*negotiate_rates)(struct clk_hw *hw,
                        struct clk_rate_request *req,
                        bool (*check_rate)(struct clk_core *,
                                           unsigned long));
```

The **`check_rate`** function pointer is a function that exists in the clk core.

Example negotiate rates for a divider

```
static bool clk_dummy_div_negotiate_rates(struct clk_hw *hw,
                                          struct clk_rate_request *req,
                                          bool (*check_rate)(struct clk_core *,
                                                             unsigned long))
{
        unsigned long rate;

        for (int i = 0; i < BIT(CLK_DUMMY_DIV_WIDTH + 1); i++) {
                rate = divider_recalc_rate(hw, req->best_parent_rate, i, NULL,
                                           CLK_DIVIDER_ROUND_CLOSEST,
                                           CLK_DUMMY_DIV_WIDTH);
                if (check_rate(req->core, rate))
                        return true;
        }

        return false;
}
```

New code in clk core uses the return value to pick the first rate that's acceptable to all nodes in the subtree.

kunit tests with v1/v2 testing

Current patch set has a kunit test case that checks for the expected behavior with the v1 and v2 rate negotiation logic.

```
// Test setup for clock tree not shown for simplicity
ret = clk_set_rate(ctx->child1_clk, 48 * HZ_PER_MHZ);
KUNIT_ASSERT_EQ(test, ret, 0);

KUNIT_EXPECT_EQ(test, clk_get_rate(ctx->parent_clk), 48 * HZ_PER_MHZ);

KUNIT_EXPECT_EQ(test, clk_get_rate(ctx->child1_clk), 48 * HZ_PER_MHZ);
KUNIT_EXPECT_EQ(test, ctx->child1.div, 1);

if (clk_use_v2_rate_negotiation(ctx->child1_clk)) {
        KUNIT_EXPECT_EQ(test, clk_get_rate(ctx->child2_clk), 24 * HZ_PER_MHZ);
        KUNIT_EXPECT_EQ(test, ctx->child2.div, 2);
} else {
        // Legacy behavior in v1 logic where sibling clks are unexpectedly changed.
        KUNIT_EXPECT_EQ(test, clk_get_rate(ctx->child2_clk), 48 * HZ_PER_MHZ);
        KUNIT_EXPECT_EQ(test, ctx->child2.div, 1);
}
```